MP 5149 · UT5 · INTRODUCCIÓ A LES XARXES NEURONALS SUPERVISADES

Del perceptró al perceptró multicapa

Com funciona una neurona artificial, per què una sola no basta, i com una xarxa aprèn a construir ella mateixa les característiques que necessita.

Desenvolupament de components software per a sistemes d'aprenentatge automàtic · Politècnic Llevant · Curs 2026-27

ABANS DE COMENÇAR

Dos mòduls en paral·lel, dues famílies d'algorismes

MP 5134

Algorismes clàssics

Disseny i avaluació de models

Regressió lineal i logística, k-nearest neighbors, decision trees, random forest, SVM, k-means.

Cada algorisme és **una idea concreta** sobre com separar o ajustar les dades. Els feu servir amb scikit-learn i us centreu a **avaluar-los**.

MP 5149

Xarxes neuronals

Desenvolupament de components software

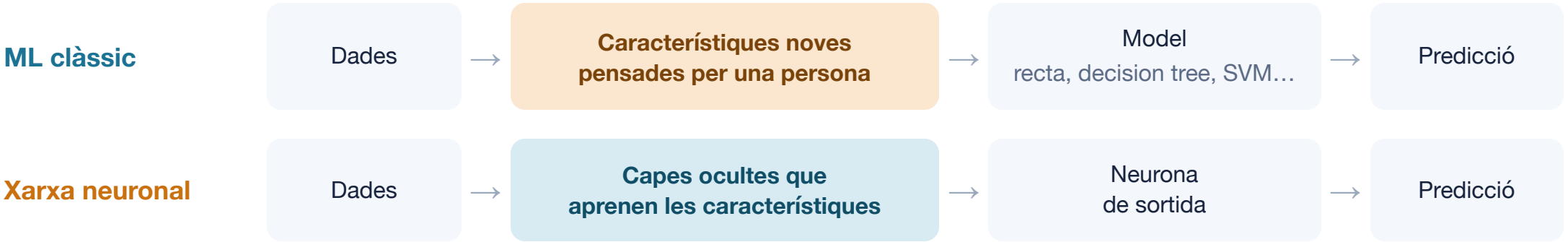
Perceptró, perceptró multicapa, mapes autoorganitzatius, xarxes de Hopfield.

Totes es construeixen igual: **moltes neurones molt simples, totes iguals, connectades en capes**. Cap neurona sola fa gran cosa; la potència ve de connectar-ne moltes.

Als dos mòduls es treballa igual: amb scikit-learn, qualsevol model s'entrena amb `fit` i fa prediccions amb `predict`. Aquí no programarem les xarxes per dins ni en farem les fórmules: les **entrenarem amb biblioteques** (scikit-learn i PyTorch) i n'**observarem** el comportament.

LA DIFERÈNCIA QUE FARÀ DE FIL CONDUCTOR

Qui fabrica les característiques?



> Als algorismes clàssics, les característiques les prepares tu. A les xarxes neuronals, les capes ocultes les aprenen.

Ara sona abstracte. D'aquí a unes quantes diapositives ho veurem passar amb quatre punts.

DUES FAMÍLIES

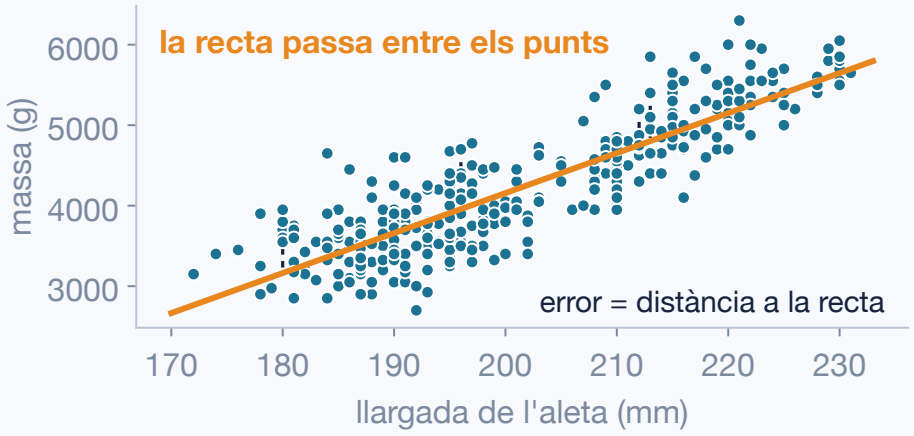
En què es diferencien

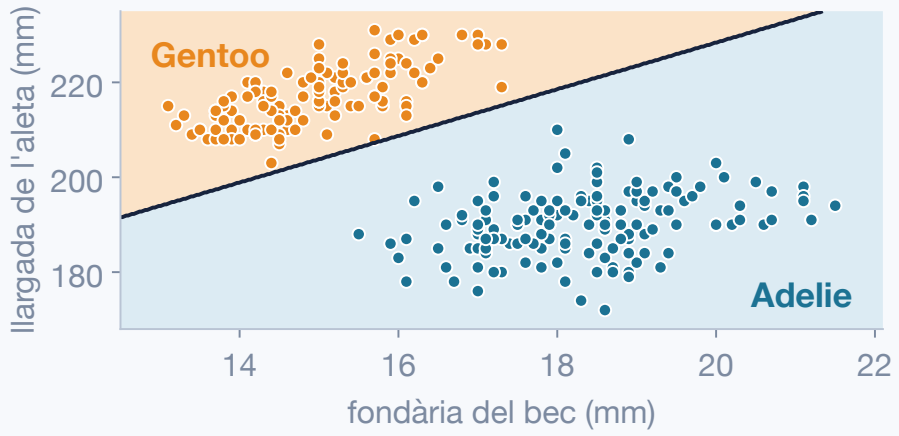
	ALGORISMES CLÀSSICS	XARXES NEURONALS
Què són	Una idea per algorisme: una recta, preguntes encadenades, el marge més ample	Moltes neurones iguals i simples, connectades en capes
Característiques	Les prepara la persona: polinòmiques, transformacions, codificació	Les capes ocultes aprenen representacions noves
Entrenament	Un mètode específic per a cada algorisme; molts donen sempre el mateix resultat	Sempre la mateixa recepta: gradient descent i backpropagation. Depèn de l'atzar inicial
On brillen	Taules de dades estructurades	Imatges, so, text: dades no estructurades i abundants
Cost	Segons, en qualsevol ordinador	Sovint necessiten GPU
Explicació	Molts es poden llegir: coeficients, decision trees	Difícils d'explicar: milers o milions de pesos

Atenció: **no són sempre millors**. Amb dades tabulars, un random forest o un *gradient boosting* sovint iguala o supera una xarxa, i és molt més barat.

ABANS DE COMENÇAR

Regressió i classificació: dues preguntes diferents





REGRESSIÓ · EL QUE HEU FET AL 5134

Què predim	un **número**: la massa, un preu, una temperatura
La recta	**passa entre** els punts
L'error	**a quina distància** queda cada punt
scikit-learn	`LinearRegression, MLPRegressor`

CLASSIFICACIÓ · EL QUE FAREM EN AQUESTA UNITAT

Què predim	una **classe**: l'espècie, el dígit, sí o no
La recta	**separa** els punts en dos grups
L'error	el punt cau **al costat equivocat**
scikit-learn	`Perceptron, LogisticRegression, MLPClassifier`

La neurona sempre calcula la mateixa suma z. En regressió, **z ja és la resposta**; en classificació, z es converteix en una **classe** o una probabilitat. Per això la **regressió logística**, malgrat el nom, **classifica**. Aquesta unitat és de classificació (la regressió amb xarxes, al NB 1.1, secció 11.1).

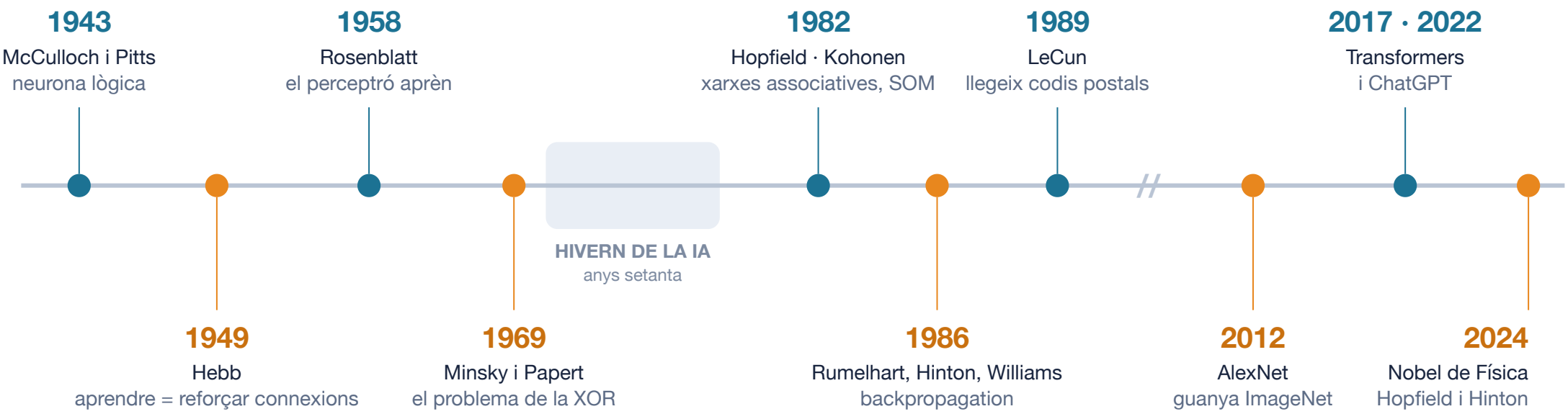
PART 1

Vuitanta anys d'eufòria i d'hiverns

La història de les xarxes neuronals és la d'un problema que ningú sabia resoldre, i de la idea que el va desbloquejar.

1 · HISTÒRIA

Una línia del temps amb alts i baixos

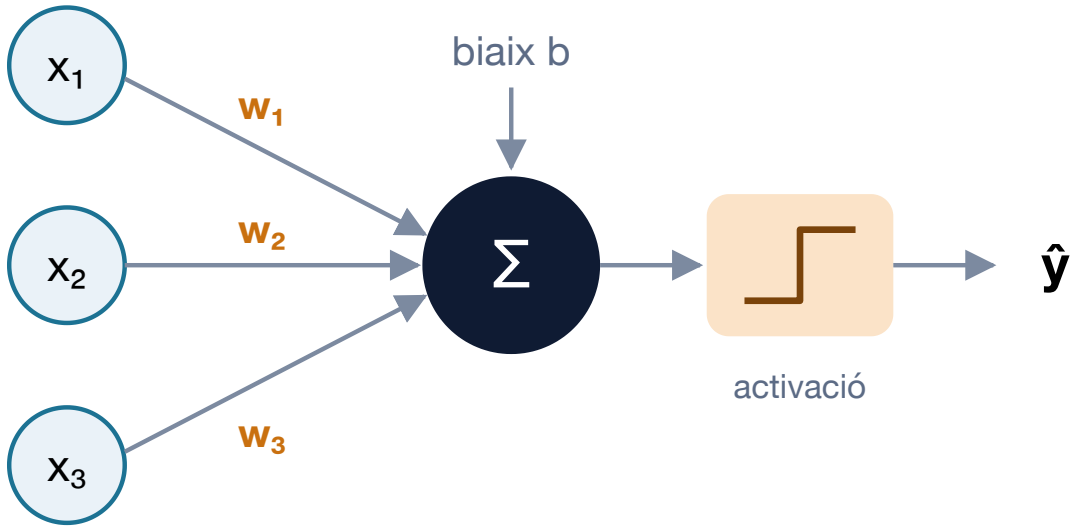


Avui ens quedam entre 1943 i 1986. Hopfield i Kohonen són la UT7 d'aquest mòdul.

De la neurona biològica a l'artificial

BIOLOGIA	NEURONA ARTIFICIAL
Senyals que arriben per les **dendrites**	**Entrades** x_1, x_2, x_3
Força de cada **sinapsi**	**Pesos** w_1, w_2, w_3
El **soma** suma els senyals	**Suma ponderada**
Es dispara si supera un **llindar**	**Biaix** i **funció d'activació**
Aprendre = reforçar sinapsis	**Entrenar = ajustar pesos**

El cervell té uns 86.000 milions de neurones. Les xarxes artificials se n'**inspiren**, però no el simulen.



CLASSIFICACIÓ · 2 CLASSES

McCulloch i Pitts: una neurona que calcula

Entrades de 0 o 1. La neurona compta quantes n'hi ha d'actives i, si arriba al **llindar**, dispara.

Canviant només el llindar, la mateixa neurona fa portes lògiques diferents. I amb AND, OR i NOT es pot construir **qualsevol càlcul**.

Però no aprèn res. El llindar el tries tu. És programació clàssica amb forma de neurona: `DADES + REGLES → RESPOSTES`

x_1	x_2	AND llindar 2	OR llindar 1
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	1

CLASSIFICACIÓ · 2 CLASSES

Rosenblatt: el perceptró

$$z = w_1 \cdot x_1 + w_2 \cdot x_2 + b$$

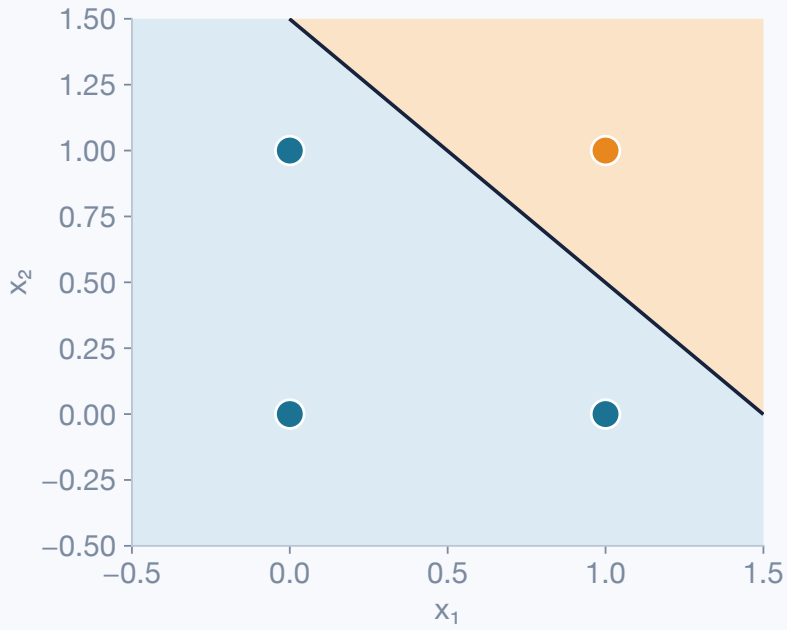
$$\hat{y} = 1 \text{ si } z \geq 0, \text{ si no } 0$$

1. Una **suma ponderada** de les entrades, més el biaix. 2. Una **funció d'activació**, la funció step, que converteix z en 0 o 1. I la novetat respecte de 1943: **els pesos s'aprenen a partir d'exemples**.

Amb $\mathbf{w_1 = 1}$, $\mathbf{w_2 = 1}$ i $\mathbf{b = -1{,}5}$, $z = x_1 + x_2 - 1{,}5$:

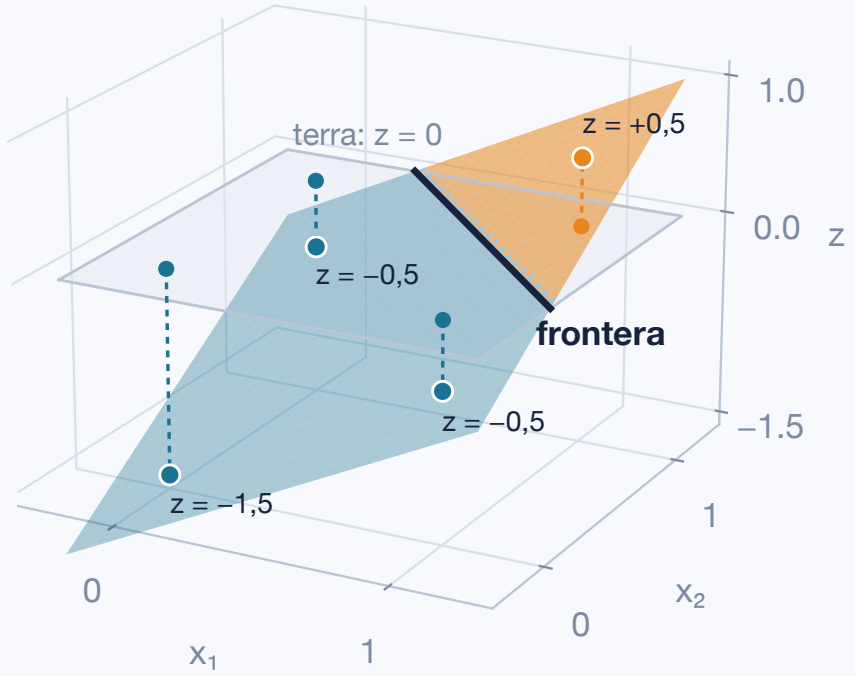
x_1, x_2	z	$z \geq 0$?	$\hat{y}$
0, 0	−1,5	no	0
0, 1	−0,5	no	0
1, 0	−0,5	no	0
1, 1	+0,5	sí	**1**

És la porta **AND**.



La línia negra són els punts on **z = 0**: per damunt la neurona diu 1, per davall diu 0. Els **pesos** inclinen la recta i el **biaix** la desplaça. **Entrenar un perceptró és buscar la recta** que deixa cada classe a un costat.

Amb dues entrades, la recta és un pla



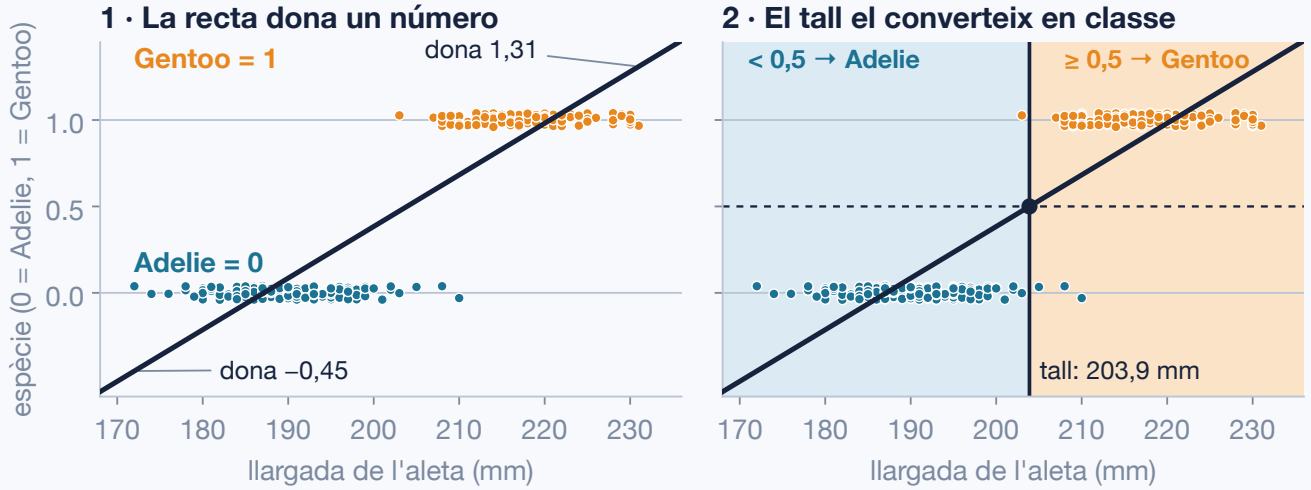
La porta AND d'abans: $z = x_1 + x_2 – 1{,}5$. Amb **dos pesos i un biaix**, z és un **pla inclinat**: cada punt (x_1, x_2) en té l'altura.

Allà on el pla **talla el terra** (z = 0) hi ha la **frontera**: la recta de la diapositiva anterior. Per damunt del terra, la neurona diu 1; per davall, 0.

Amb una entrada, la frontera és un punt; amb dues, una recta; amb tres, un pla. La idea sempre és la mateixa.

CLASSIFICACIÓ · 2 CLASSES

Una recta i un llindar



Pensa en un examen. La **nota** és un número; **aprovat** o **suspès** és una classe. Per passar de l'un a l'altre, un **tall**: el 5.

Una neurona fa el mateix. Ho veiem amb una sola mesura, l'aleta, per distingir Adelie (0) de Gentoo (1).

1 · La recta dona un número per a cada pingüí: és una regressió lineal, com al 5134. Però –0,45 o 1,31 no són cap espècie.

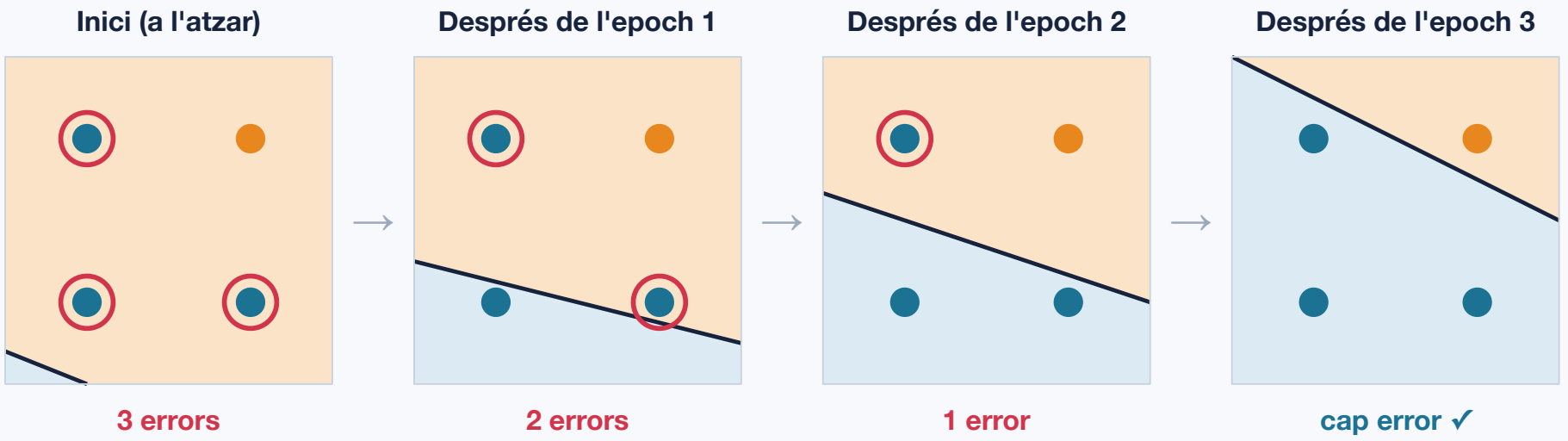
2 · El tall a 0,5 el converteix en classe: ≥ 0,5 → Gentoo. La frontera és un punt, **203,9 mm**, i encerta el 98,5%.

Això és una neurona: una recta (z) i una **funció d'activació**. Al perceptró, l'activació és aquest tall: la **funció step**.

CLASSIFICACIÓ · 2 CLASSES

La regla del perceptró

Comença amb una recta qualsevol i repassa els punts un a un. **Si encerta, no toca res.** Si s'equivoca, **acosta una mica la recta a aquell punt**, per intentar deixar-lo al costat bo. Una passada per tots els punts és una **epoch**.



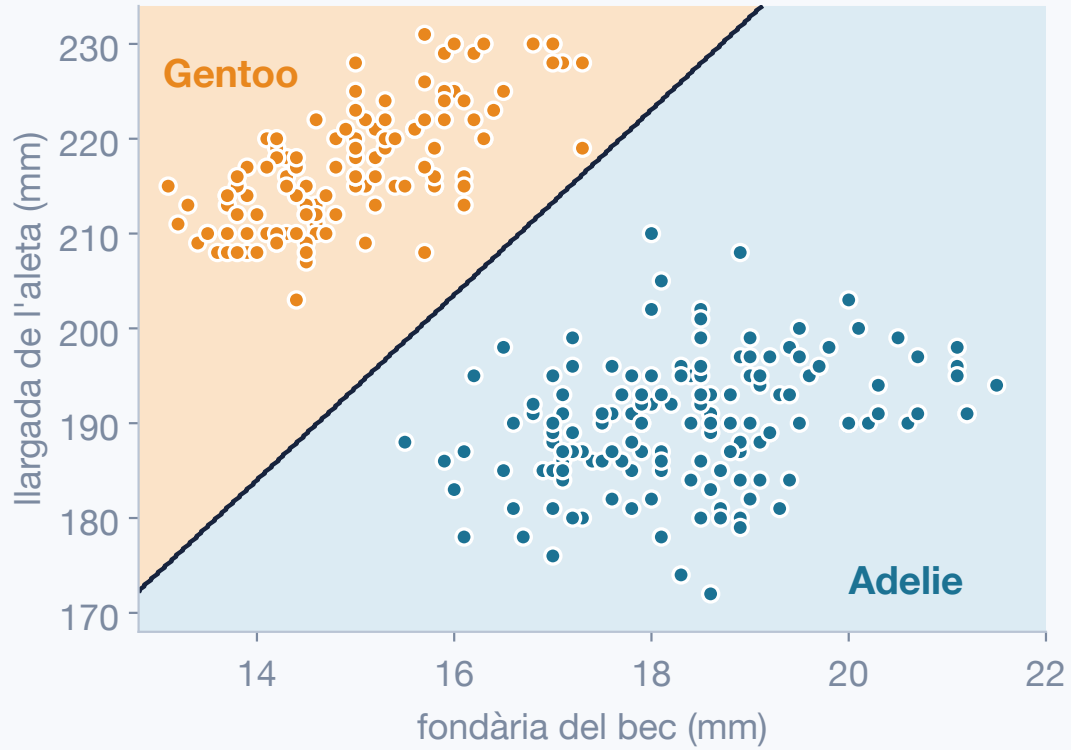
La porta AND. Color del punt: la resposta correcta. Color del fons: el que diu la neurona. Encerclats en vermell, els errors.

Cada moviment és petit. **Com** de petit ho diu el **learning rate** (`Perceptron(eta0=0.1)`): és el primer **hiperparàmetre**. No s'aprèn, el tries tu.

Teorema de convergència: si existeix una recta que separa les classes, el perceptró la troba en un nombre finit de passos. Fixau-vos en el «si».

CLASSIFICACIÓ · 2 CLASSES

Adelie o Gentoo, amb una sola neurona



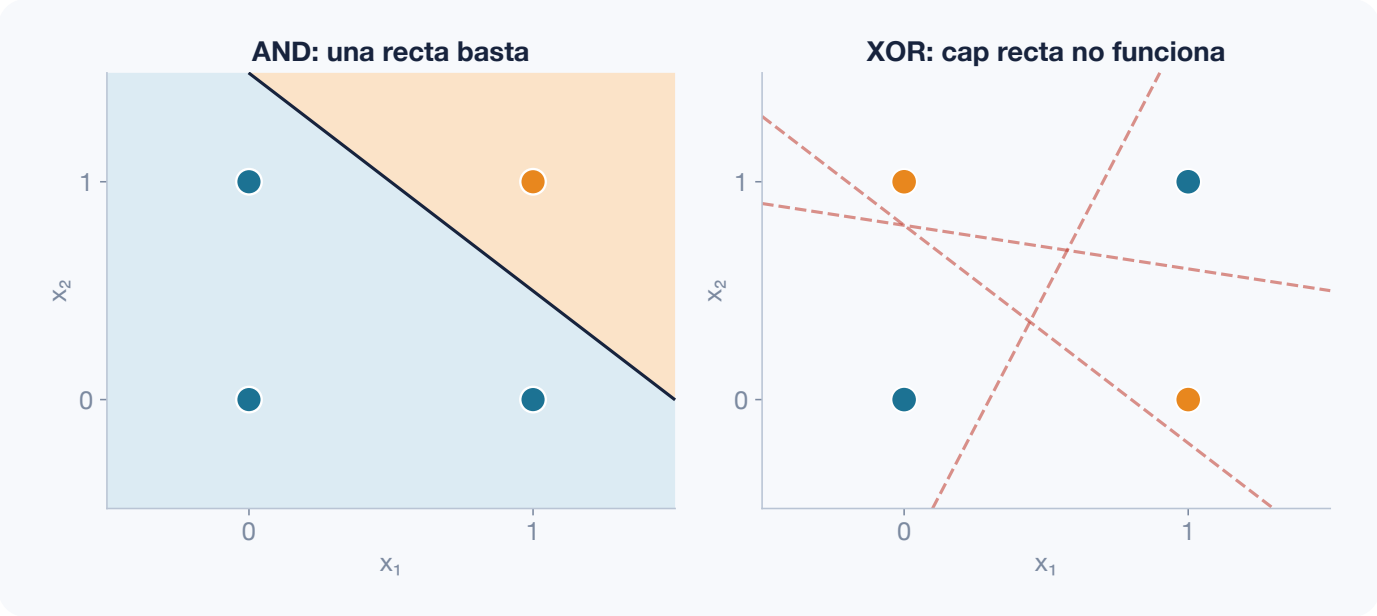
274 pingüins del 5134. Dues mesures. **Encert a test: 100%**.

Però l'escala de les entrades importa. Amb les mesures en mm, el perceptró s'equivoca en gairebé la meitat dels pingüins durant les dues primeres epochs; amb les mesures escalades, ho resol a la primera.

L'aleta fa ~200 mm i el bec ~17: les correccions d'un pes són deu vegades les de l'altre. **A les xarxes neuronals, escalar no és opcional**: `StandardScaler` + perceptró en una pipeline.

CLASSIFICACIÓ · 2 CLASSES

El mur de la XOR



XOR val 1 quan **una i només una** entrada és 1.

Els punts taronges són a una diagonal i els blaus a l'altra: **no hi ha cap recta** que els separi. El perceptró hi pot estar epochs i epochs.

Minsky i Papert ho van analitzar al llibre *Perceptrons*. La retallada de finançament que va seguir es coneix com el **primer hivern de la IA**.

Com ho resoldria un algorisme clàssic?

x_1	x_2	$x_1 \cdot x_2$	z	ŷ	XOR
0	0	**0**	−0,5	0	0
0	1	**0**	+0,5	1	1
1	0	**0**	+0,5	1	1
1	1	**1**	**−0,5**	0	0

$$z = x_1 + x_2 - 2 \cdot (x_1 \cdot x_2) - 0{,}5$$

Però algú ha hagut de **pensar** que el producte era la columna que calia. Amb quatre punts és fàcil. Amb una foto d'un milió de píxels, quina columna inventes per saber si hi ha un gat?

Al 5134, quan una recta no basta, **fabricam una característica nova**. Aquí, la columna **$x_1 \cdot x_2$**: només val 1 a la fila (1, 1), just la que fa nosa.

Ara el perceptró té tres entrades i pot fer això: **compta els uns** ($x_1 + x_2$), però si n'hi ha dos, la columna nova **en resta dos**. La fila (1, 1) passa de z = 1,5 a z = −0,5 i cau al costat del 0. **Les quatre files, bé.**

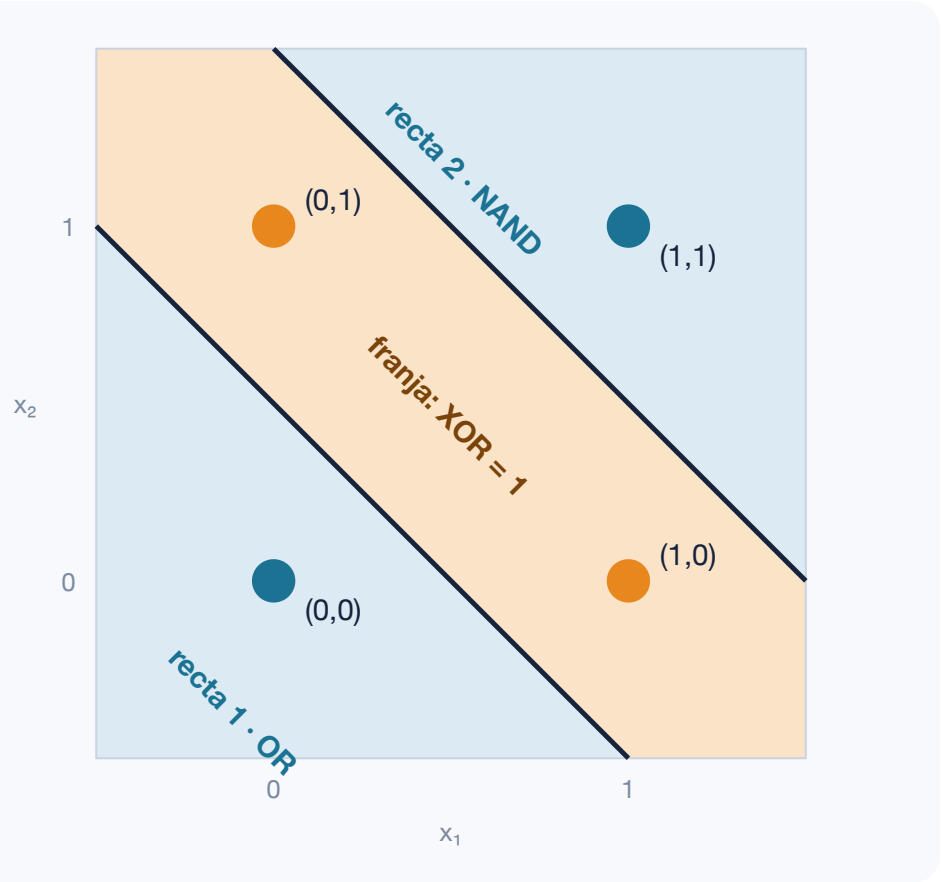
> I si la xarxa aprengués sola quines característiques necessita?

PART 2

La solució: afegir capes

Posar neurones entre l'entrada i la sortida. Primer ho farem a mà, per entendre per què funciona. Després, que ho aprengui sola.

Si una recta no basta, en posam dues



Els taronges són **al mig**, entre els dos blaus. Una recta talla el pla en dos trossos; per aïllar la franja del mig en calen **dues**.

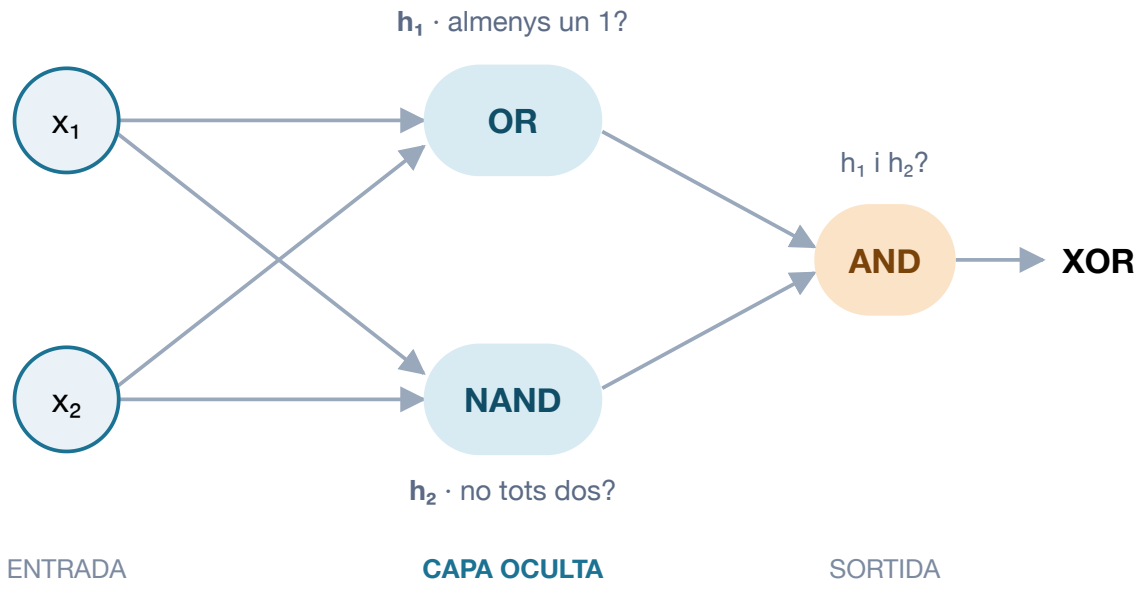
RECTA	QUÈ PREGUNTA	PORTA
1	Hi ha **almenys un** 1? Deixa fora el (0, 0).	**OR**
2	Que **no** siguin **tots dos** 1? Deixa fora el (1, 1).	**NAND**

XOR = 1 quan les dues respostes són **sí**: el punt és dins la franja.

Cada recta és un perceptró normal, com el de l'AND. Ja sabem fer-los. Només en falta un tercer que en combini les respostes.

CLASSIFICACIÓ · 2 CLASSES

La XOR amb una capa oculta

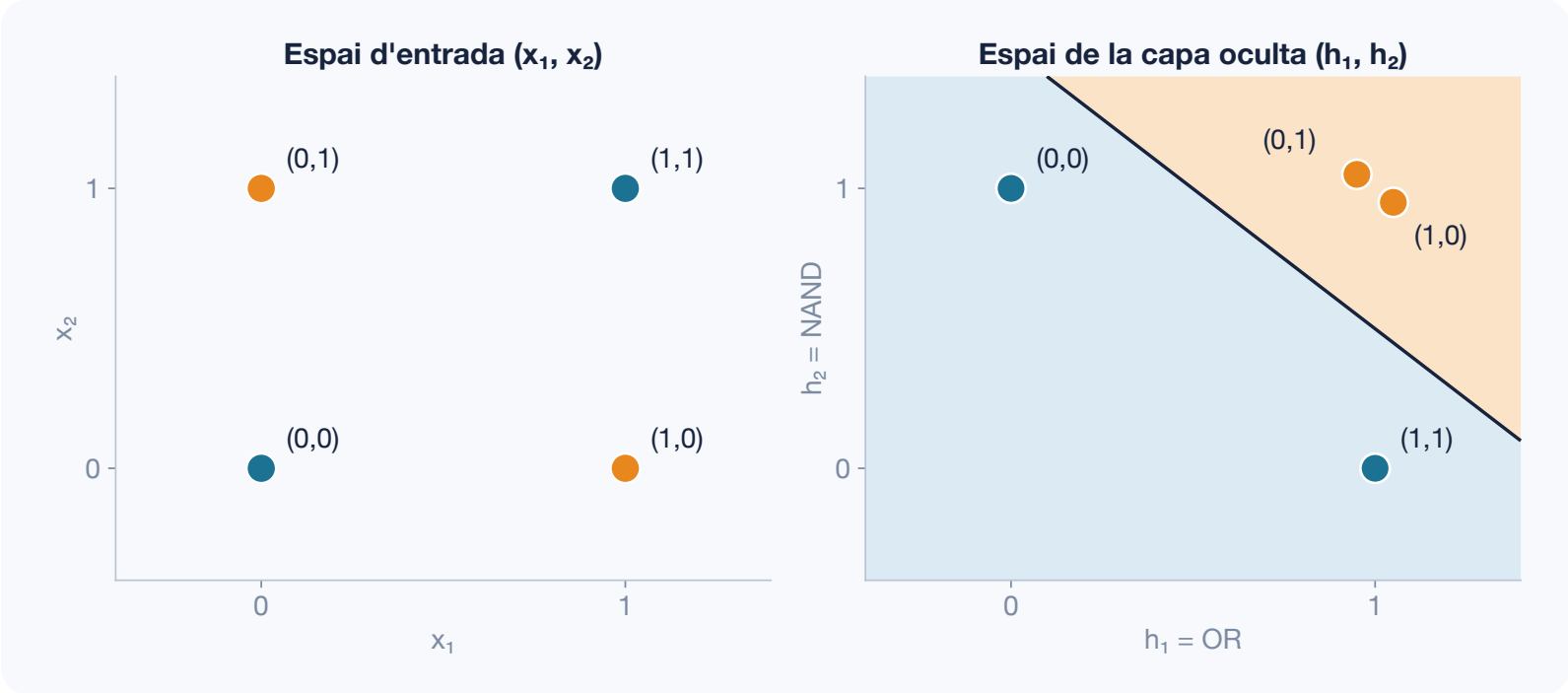


Seguim les quatre files per la xarxa:

x_1	x_2	h_1 · OR	h_2 · NAND	AND
0	0	0	1	0
0	1	1	1	1
1	0	1	1	1
1	1	1	0	0

La sortida és exactament la XOR. Les neurones del mig no es veuen ni a l'entrada ni a la sortida: formen una **capa oculta**. Això ja és un **perceptró multicapa** (MLP).

La capa oculta transforma el problema



La neurona de sortida **no veu x_1 ni x_2**: només veu h_1 i h_2, les columnes de la taula anterior. Si les feim servir de coordenades, **els mateixos quatre punts** canvien de lloc (cada un duu l'etiqueta de la seva entrada): els dos taronges cauen junts a (1, 1).

I dir 1 només a (1, 1) és la **porta AND** de la diapositiva 10: una recta basta. h_1 i h_2 són **característiques noves**, com $x_1 \cdot x_2$. La diferència: les fa una capa de neurones i, en una xarxa entrenada, **les tria l'entrenament**.

Per què va costar disset anys?

Saber que les capes funcionen no diu **com trobar-ne els pesos**. Hi havia dos obstacles.

1. A qui tiram la culpa?

La regla del perceptró compara la sortida amb la resposta correcta. Però una neurona oculta **no té resposta correcta**: ningú no li ha dit que havia de fer OR.

Si la xarxa s'equivoca, quina neurona oculta en té la culpa, i quanta?

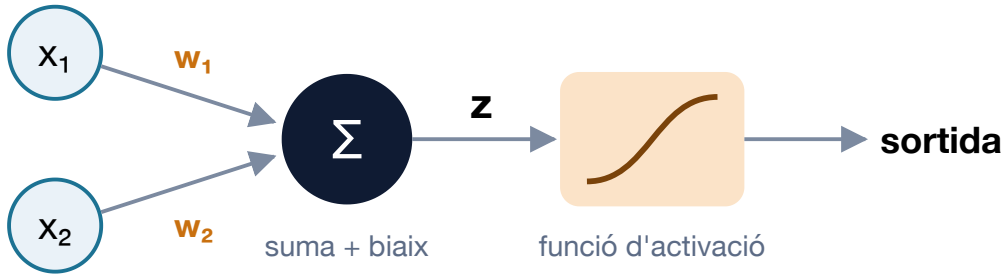
2. La funció step és plana

La neurona dona 0 o 1 i res entremig. Si mous un pes una mica, la sortida continua sent la mateixa: **no canvia gens**.

Sense cap canvi, no hi ha pista de cap a on moure el pes per millorar.

La solució té tres peces: **funcions d'activació sense salts**, **gradient descent** i **backpropagation**.

Què fa la funció d'activació?



Cada neurona fa dues passes: calcula **z**, la suma ponderada més el biaix, i la passa per la **funció d'activació**, que decideix què treu la neurona.

Un interruptor

La **step**: apagat o encès, res entremig.

Un regulador de llum

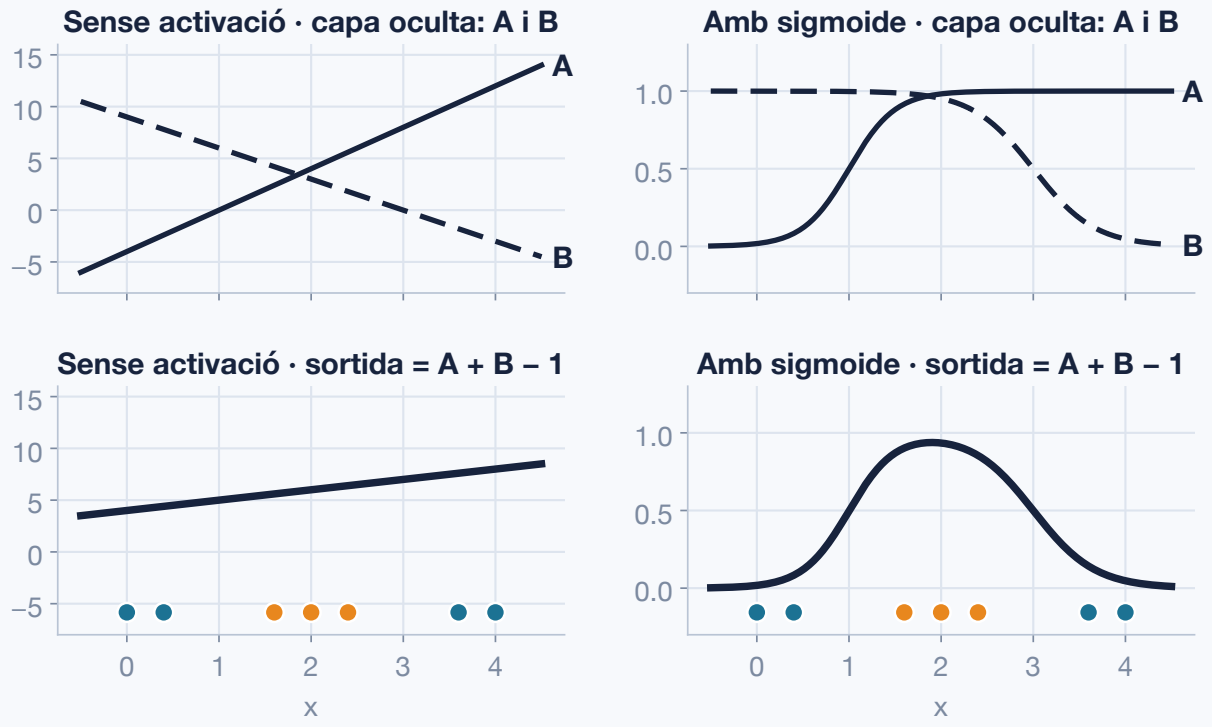
La **sigmoide**: com més z, més llum, de mica en mica.

z →	−2	−1	0	1	2
step	0	0	1	1	1
sigmoide	0,12	0,27	0,50	0,73	0,88
tanh	−0,96	−0,76	0	0,76	0,96
ReLU	0	0	0	1	2
identitat (cap)	−2	−1	0	1	2

La step salta de cop entre −1 i 0. La sigmoide fa el mateix camí, de prop de 0 a prop de 1, però **passant per tots els valors del mig**. La identitat no fa res: és com no posar-hi activació.

Una funció d'activació té **dues feines**: **doblegar** la recta i **no fer salts**.

Sense activació, les capes no serveixen



La mateixa xarxa i **els mateixos pesos**: una entrada x, dues neurones ocultes (A i B) i una sortida que les suma. Només canvia l'activació.

Sense activació, A i B són rectes, i sumar rectes dona una altra recta. Només talla una vegada: no pot aïllar els punts taronja del mig. Amb cent capes, igual.

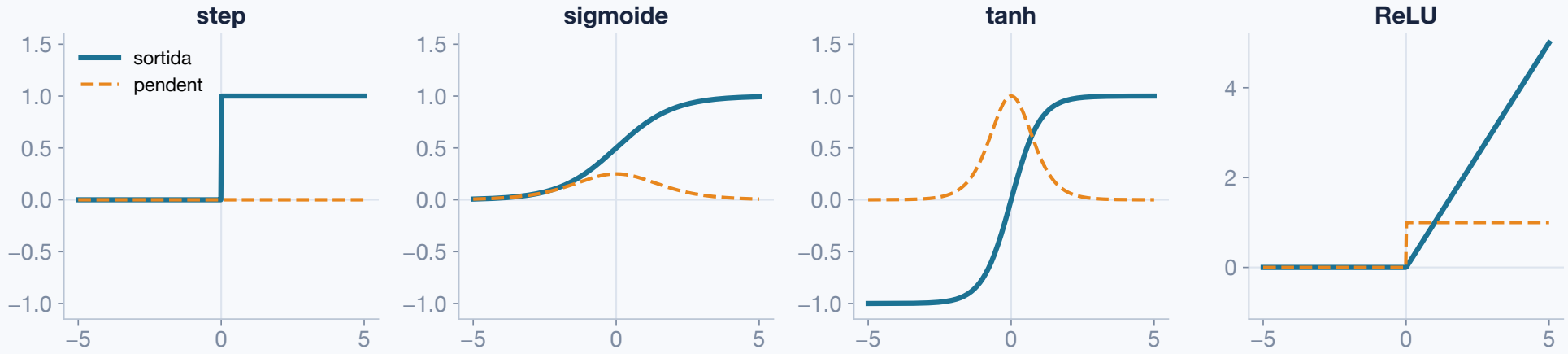
Amb sigmoide, A s'encén a x = 1 i B s'apaga a x = 3. La suma fa una **muntanyeta** just on hi ha els taronja: la franja de la XOR, en una dimensió.

Amb moltes muntanyetes, una xarxa pot dibuixar **qualsevol corba**.

Per al 5134: una neurona amb sigmoide és exactament una **regressió logística**. I una xarxa sense activació també, per moltes capes que tingui.

Funcions d'activació sense salts

La step salta de cop: el seu **pendent** és zero a tot arreu i no dona cap pista. Les altres tres pugen de mica en mica.



Fred o calent. Una neurona hauria de dir 1 i té z = –2. Movem una mica un pes i z passa a –1,5. Hem millorat?

step	0 → 0	cap pista
sigmoide	0,12 → 0,18	**«calent»: per aquí**

Aquesta pista és el **pendent**. El gradient descent el farà servir per decidir cap a on moure cada pes.

	DOBLEGA	SENSE SALTS	ON LA TROBAREM
identitat	**no**	sí	a la sortida d'una regressió
step	sí	**no**	al perceptró
sigmoide	sí	sí	a la sortida: es llegeix com una probabilitat
tanh	sí	sí	a la capa oculta, al NB 1.1
ReLU	sí	sí	per defecte a les xarxes profundes

Loss i gradient descent

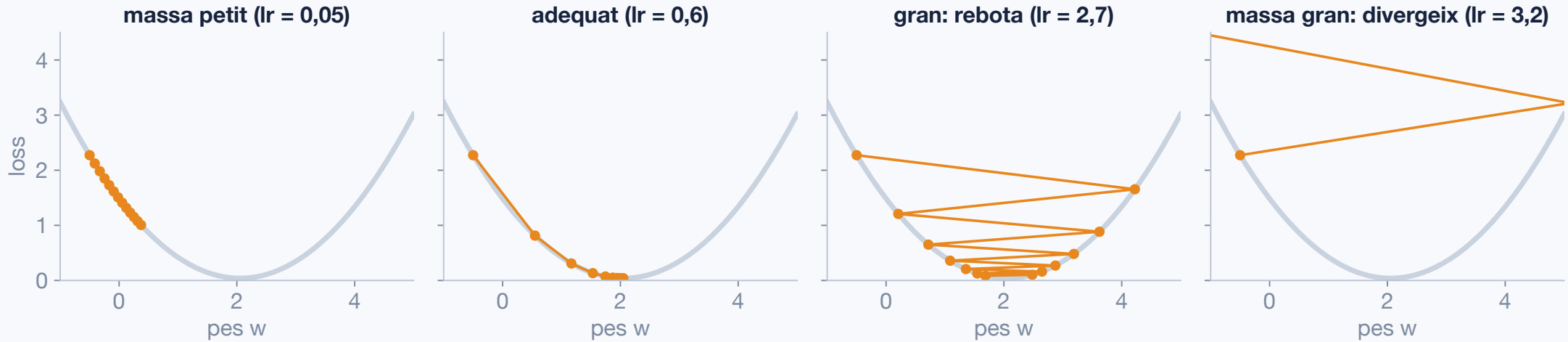
La **loss** diu com de malament ho fa la xarxa. Al 5134 vau veure que dibuixa una **vall**.

Enmig de la boira, per baixar mires cap a on fa pendent i fas una passa cap avall.

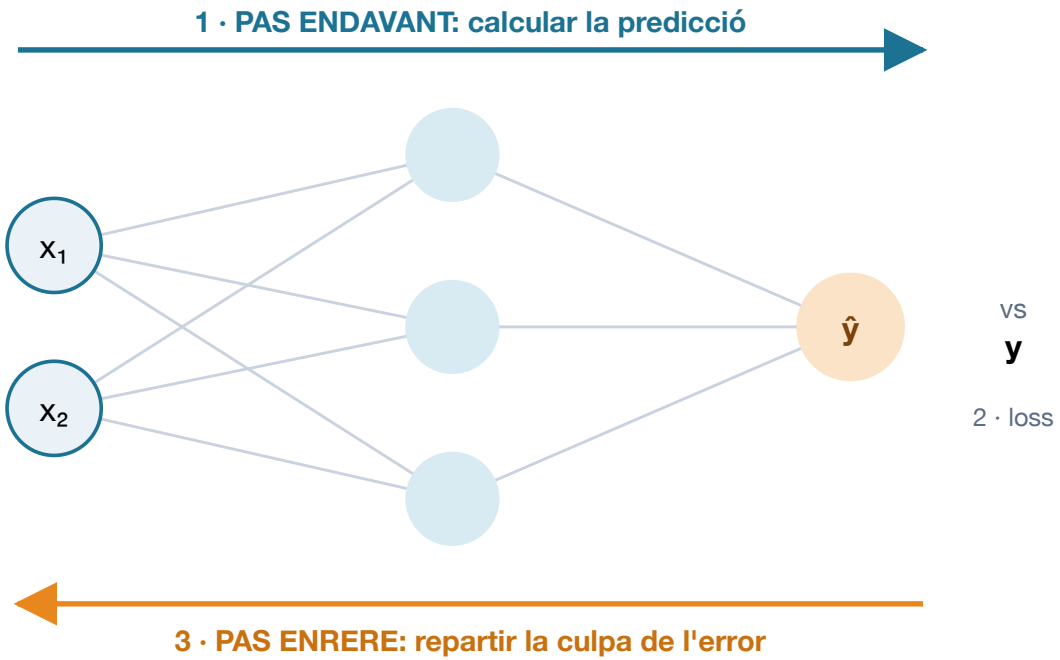
```
w ← w − learning_rate · pendent
```

pendent: el de la **loss respecte a aquest pes**, on ets ara. Cada pes té el seu, i el conjunt de tots és el **gradient**.

El signe menys mou el pes al revés del pendent: sempre **costa avall**. Primera passa del gràfic «adequat»: –0,5 – 0,6 · (–1,75) = 0,55.



Backpropagation: repartir la culpa cap enrere



La sortida sap exactament quant s'ha equivocat: **predicció – resposta**. És la mateixa idea que l'error del perceptró.

Cada neurona oculta en rep una part proporcional a:

- el **pes** que la uneix amb la sortida (si el seu vot pesava molt, té molta culpa)
- el **pendent** de la seva activació (si estava saturada, poca)

4 · **Pendent d'un pes = culpa de la neurona on arriba × valor que hi entra.** Amb aquest pendent, una passa de gradient descent a cada pes.

El bucle d'entrenament

```
a cada passa d'entrenament:
  1. pas endavant   → predicció
  2. loss           → com de malament
  3. pas enrere     → culpa de cada neurona
  4. actualització  → una passa avall
```

```python
model = MLPClassifier(hidden_layer_sizes=(3,),
                      activation="tanh")
for step in range(10):
    model.partial_fit(X, y, classes=[0, 1])
    print(model.loss_)    # 0.643 → 0.512
```

Entrenar una xarxa és **repetir aquests quatre passos milers de vegades**.

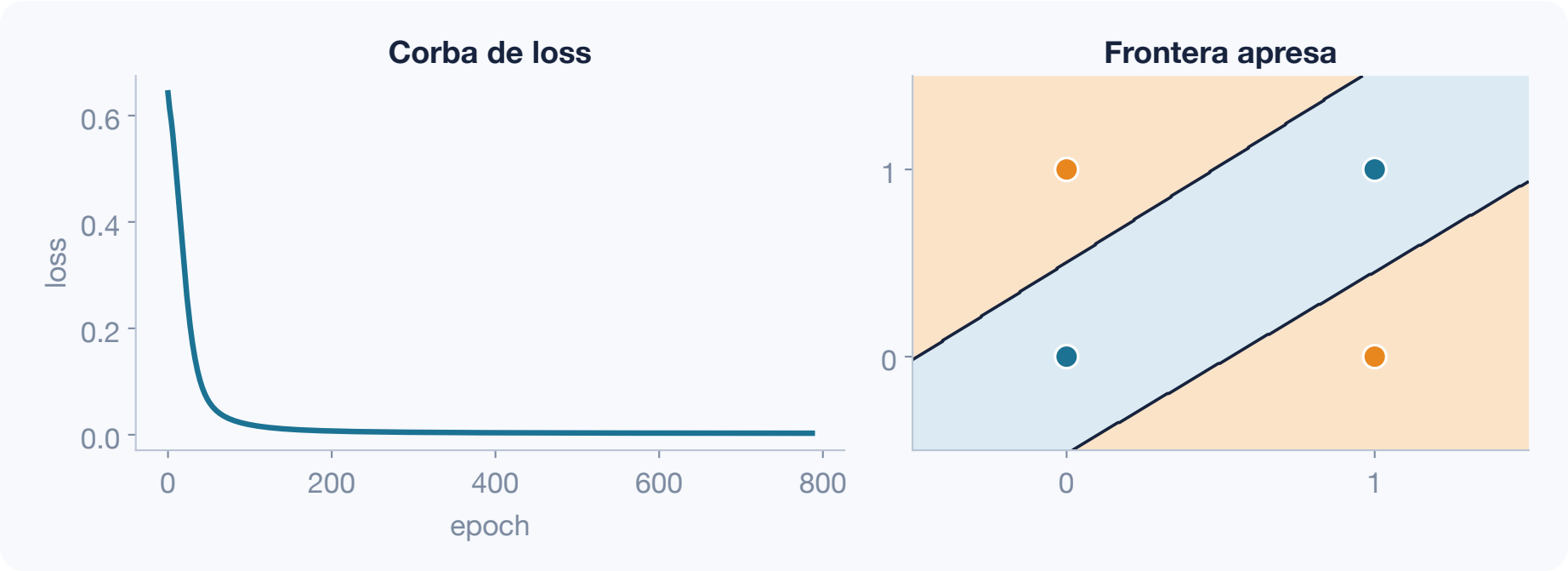
Les xarxes que reconeixen imatges o generen text fan aquest mateix bucle, amb més capes, més neurones i més dades.

No ho programem: ho observem. `MLPClassifier` fa els quatre passos cada vegada que entrena. Nosaltres en mirem el resultat: `loss_curve_`, `coefs_` i la frontera.

Animacions: els quatre passos, instància a instància, en una xarxa 2-2-1. Amb la XOR → animacio_xor_backprop.html. Amb 342 pingüins reals → animacio_pinguins_backprop.html.

CLASSIFICACIÓ · 2 CLASSES

L'MLP aprèn la XOR tot sol

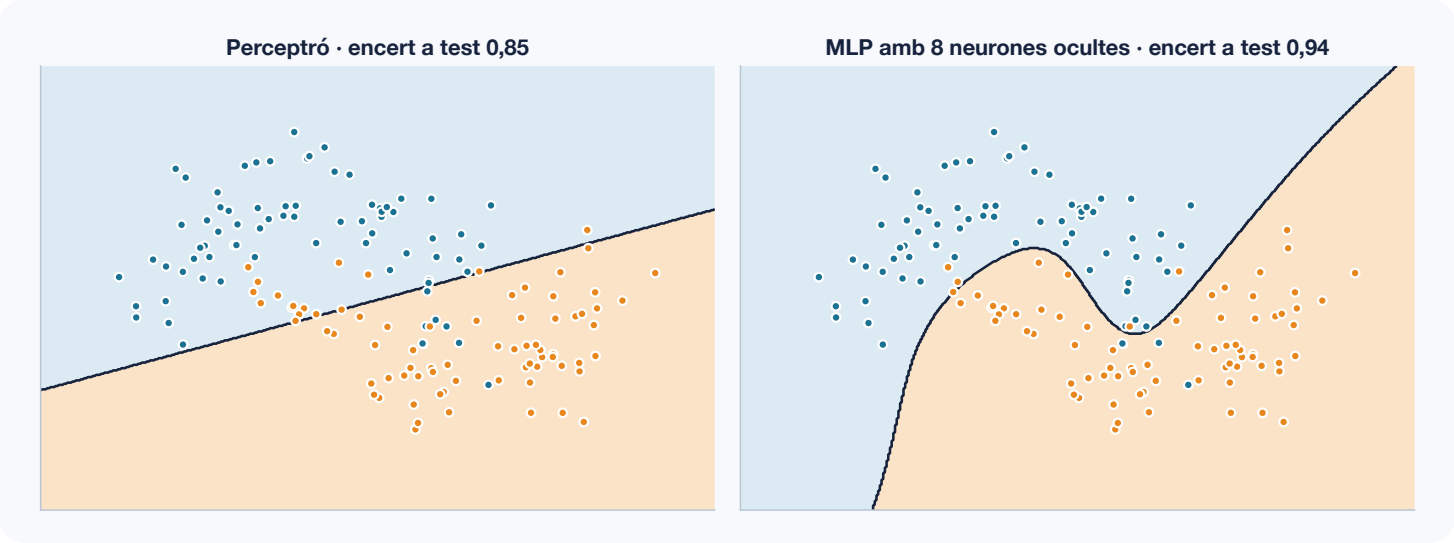


No li hem dit que calien portes OR i NAND ni quins pesos fan falta. Només les quatre files i les respostes.

La corba de loss és el primer gràfic que s'ha de mirar sempre: aquí cau en picat les primeres epochs i després s'aplana a prop de zero. I la frontera **ja no és una recta**: en són dues, que formen una franja.

CLASSIFICACIÓ · 2 CLASSES

Les dues llunes



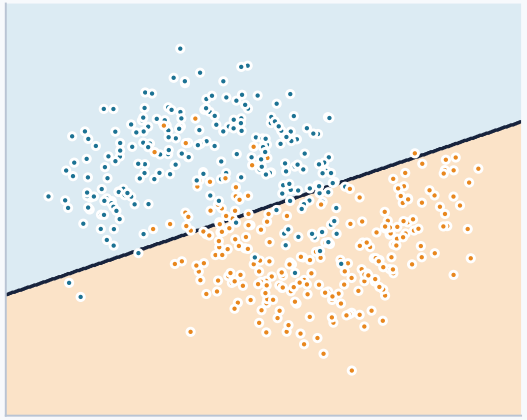
MODEL	ENCERT A TEST
Referència (Dummy)	0,50
Regressió logística	0,86
Perceptron	0,85
MLPClassifier (8, tanh)	**0,94**
MLPClassifier (16, 16)	**0,95**

Logística i perceptró són una sola neurona: una recta. No és que siguin dolents, és que el problema no és lineal.

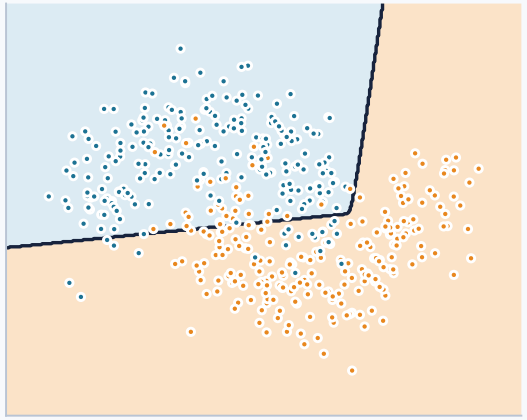
CLASSIFICACIÓ · 2 CLASSES

Neurones, learning rate i atzar

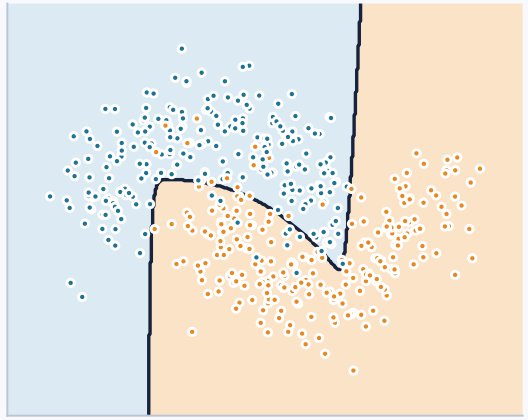
1 neurona · train 0,86 / test 0,86



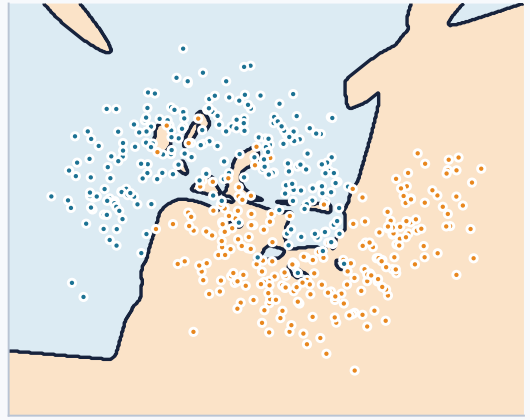
2 neurones · train 0,87 / test 0,87



4 neurones · train 0,94 / test 0,93



32 neurones · train 0,99 / test 0,92



Neurones ocultes. Amb poques, la frontera no es pot corbar. Amb 32, l'entrenament puja al 0,99 i el test baixa: memoritza el soroll, l'**overfitting**.

Learning rate. Probablement el més important. Massa petit no avança; massa gran rebota o divergeix. Es diagnostica mirant la corba de loss.

La llavor aleatòria. Amb 2 neurones ocultes, la XOR s'encalla o no segons els pesos inicials: un **mínim local**. Una regressió lineal sempre dóna el mateix.

CLASSIFICACIÓ · 2 CLASSES

MLPClassifier: cada paràmetre té sentit

```
model = MLPClassifier(
    hidden_layer_sizes=(16, 16),
    activation="tanh",
    learning_rate_init=0.01,
    batch_size=200,
    max_iter=2000,
    random_state=42)
model.fit(X_train, y_train)
model.score(X_test, y_test)
model.loss_curve_
```

PARÀMETRE	QUÈ ÉS
`hidden_layer_sizes`	capes ocultes i neurones de cada una
`activation`	funció d'activació de les capes ocultes
`learning_rate_init`	learning rate
`batch_size`	mostres per passa (part 3)
`max_iter`	màxim d'epochs
`random_state`	llavor dels pesos inicials

La interfície és la de tots els models de scikit-learn: `fit`, `predict` i `score`. **Es pot intercanviar per una regressió logística o un random forest sense tocar la resta del codi.**

Per defecte fa servir ReLU i l'optimitzador **Adam**, una variant del gradient descent que ajusta la passa de cada pes. La recepta de fons és la mateixa: pas endavant, loss, pas enrere, actualització. Per predir un número en lloc d'una classe hi ha **MLPRegressor**, amb els mateixos paràmetres.

PART 3 · NB 1.2

Dades de veritat: 70.000 dígits escrits a mà

Portem l'MLP de les llunes a MNIST. Canviar d'escala obliga a introduir tres idees noves, i ens deixa veure què aprenen de debò les neurones.

CLASSIFICACIÓ · 10 CLASSES

Una imatge és una fila de 784 números



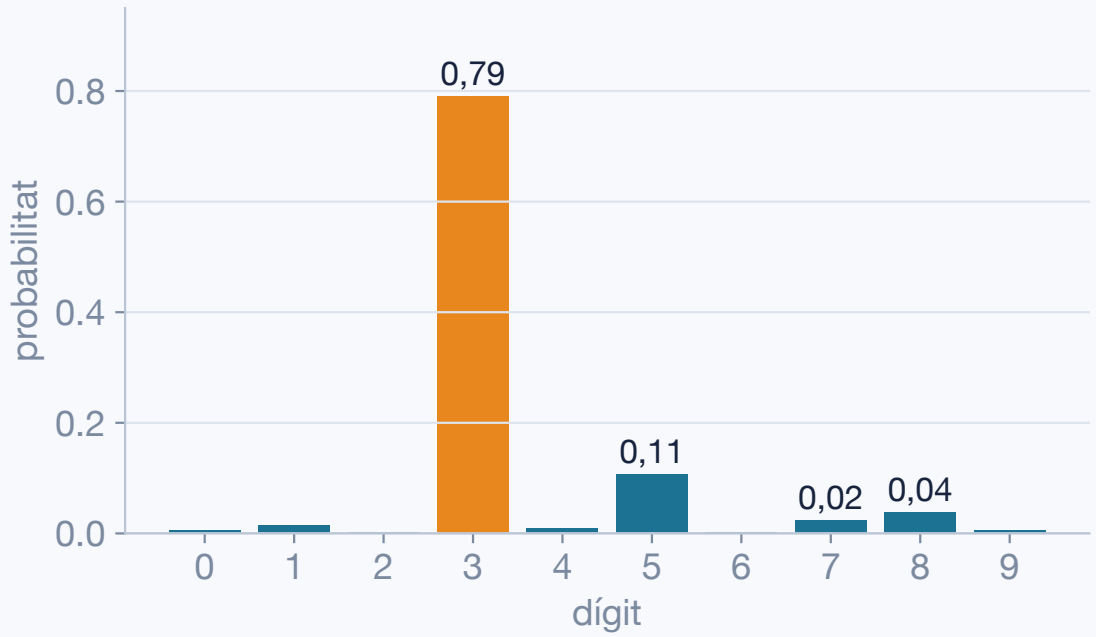
28 × 28 píxels de 0 (paper) a 255 (tinta), posats en fila: **784 columnes**.

Entrenament	50.000
Validació	10.000
Test segellat	10.000

LeCun i col·laboradors, 1998. Durant anys, *la* prova de referència de la visió per computador. A scikit-learn: `fetch_openml("mnist_784")`.

CLASSIFICACIÓ · 10 CLASSES

De 2 classes a 10: softmax



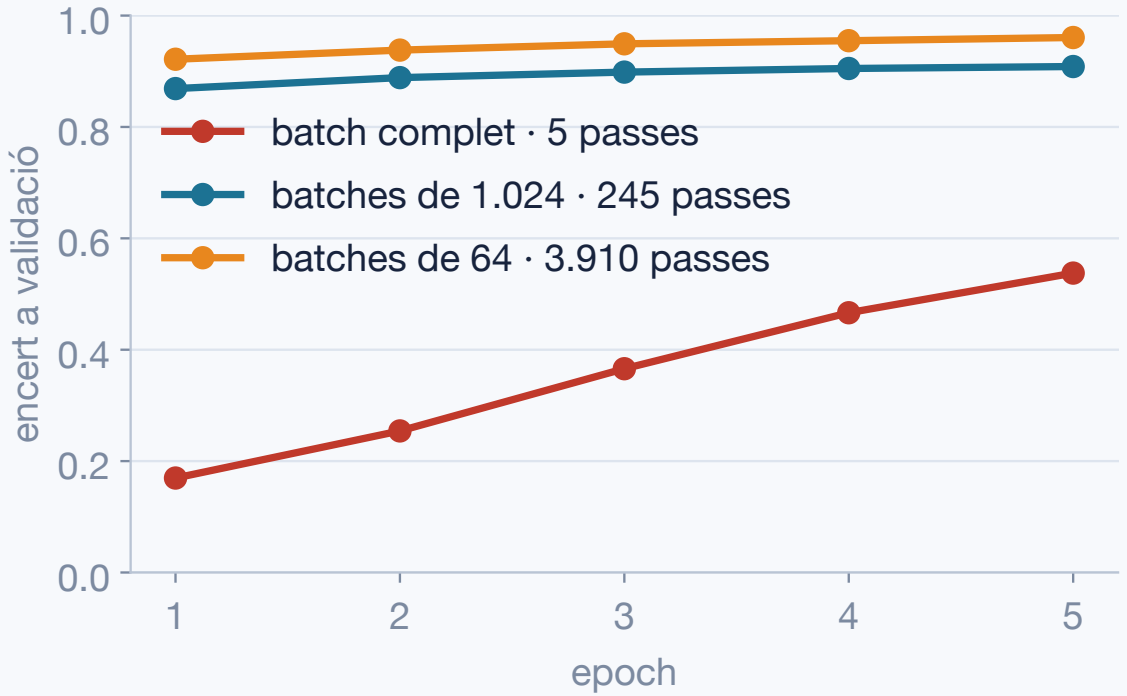
Deu neurones de sortida, una per dígit. La **softmax** converteix les deu puntuacions en deu probabilitats que sumen 1.

La resposta es codifica en **one-hot**: deu posicions, totes a 0 menys la correcta.

MLPClassifier ho fa sol. Amb més de dues classes posa la softmax a la sortida i fa el one-hot per dins; ho diu l'atribut `out_activation_`. La backpropagation és la mateixa.

Animació: la softmax i el one-hot, pingüí a pingüí, amb les tres espècies, i en 3D com la xarxa reparteix l'espai de tres mesures. S'hi poden afegir neurones ocultes → animacio_pinguins_especies.html.

Mini-batches: moltes passes per epoch



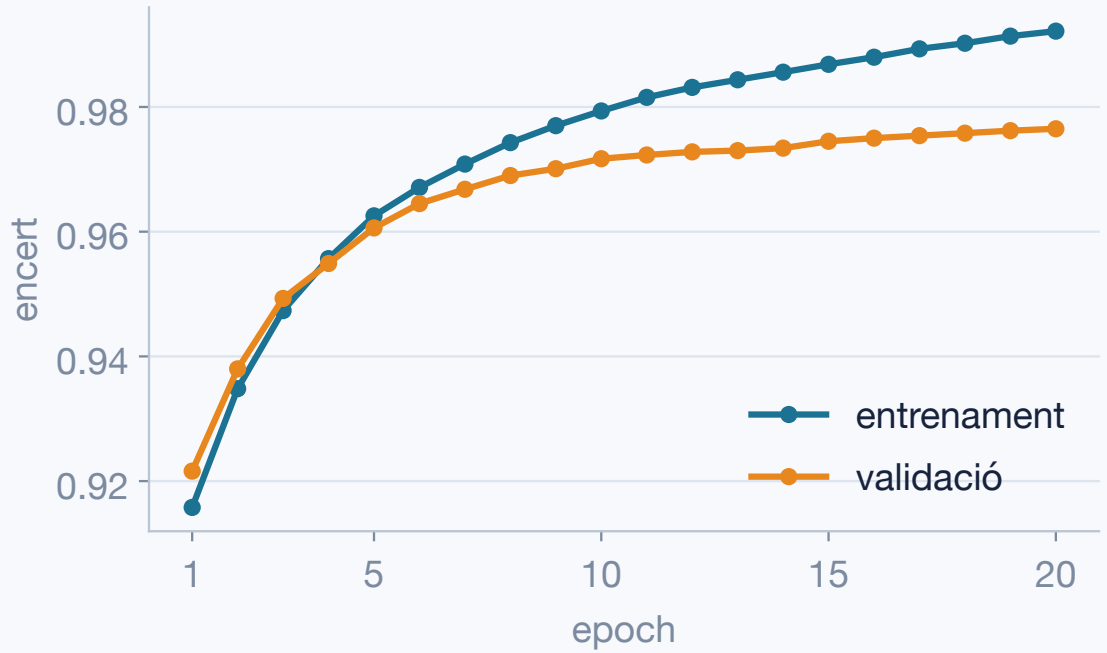
Barrejar, partir en batches de 64 i fer **una passa per batch**.

Mateixes dades i mateixes 5 epochs. Amb batches de 64 es fan quasi 800 vegades més passes, però el temps només es multiplica per tres o quatre. A scikit-learn, `batch_size`.

Iteració	un batch processat i una actualització
Epoch	totes les dades: 782 iteracions

CLASSIFICACIÓ · 10 CLASSES

Validació durant l'entrenament



Xarxa **784-128-10**: més de **100.000 pesos**. La de la XOR en tenia 13.

`MLPClassifier` amb tanh i batches de 64, entrenat epoch a epoch amb `partial_fit`: **97,65% a validació** en 20 epochs.

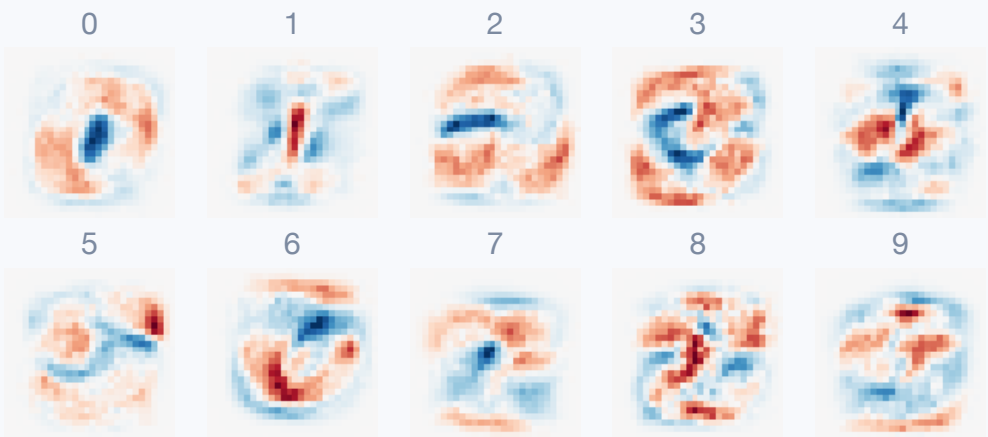
Al principi les dues corbes pugen juntes. Cap al final l'entrenament continua millorant i la validació s'atura: comença l'**overfitting**.

Test segellat, obert al final: **97,61%**. L'estimació de validació era fiable.

CLASSIFICACIÓ · 10 CLASSES

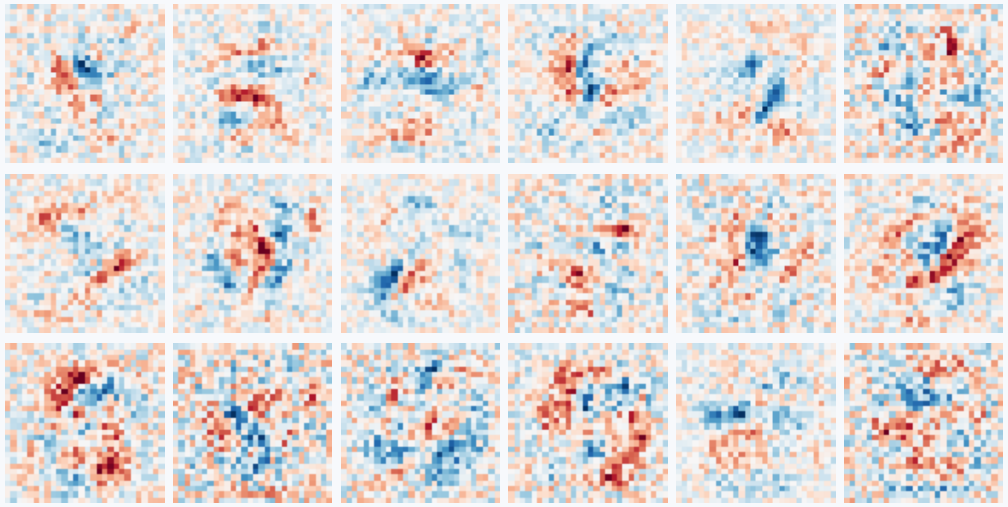
Plantilles contra traços

REGRESSIÓ LOGÍSTICA



Un model lineal aprèn **una plantilla per dígit** i hi compara cada imatge. El veureu a la UT4 del 5134.

NEURONES OCULTES DE L'MLP



Cap no és un dígit: són **traços** en una zona. La sortida combina quins traços hi ha. **Ningú no li ha dit que busqués traços.**

CLASSIFICACIÓ · 10 CLASSES

Una plantilla rígida o peces que es combinen

La mateixa capa de sortida (deu puntuacions i softmax), mirant coses diferents:

Regressió logística: mira els **784 píxels**

92,0%

Una plantilla per dígit. Funciona perquè MNIST ve centrat.

MLP: mira els **128 traços apresos** per la capa oculta

97,65%

Combina peces reutilitzables per formar cada dígit.

Errors a validació, de 10.000 imatges

802 / 235

Sense capa oculta, més del triple d'errors.

> És la figura de la XOR a gran escala: la capa oculta transforma les dades en una representació on un model lineal ja funciona.

CLASSIFICACIÓ · 10 CLASSES

Les xarxes guanyen, però per poc

MODEL	ENCERT A VALIDACIÓ
Referència (Dummy)	10,6%
Regressió logística	92,0%
Random forest (100 decision trees)	97,2%
MLP (128, tanh)	**97,65%**
MLP per defecte (ReLU, Adam)	**97,8%**

Un random forest, sense capes ocultes, queda a menys d'un punt. Fa votar molts decision trees: el veureu a la UT5 del 5134.

EL LÍMIT: DESPLAÇAR 3 PÍXELS



Regressió logística	92,0% → 44,0%
Random forest	97,2% → 54,2%
MLP (128, tanh)	97,65% → 53,6%

Per a l'MLP la imatge és una taula: **no sap quins píxels són veïns**. Les xarxes **convolucionals** reutilitzen el mateix detector a totes les posicions, i llavors la distància es fa enorme. Són la **UT6**.

Llavors, per a què serveix un MLP?

Sol i sobre píxels, avui gairebé ningú no el triaria. Però no ha desaparegut: **s'ha convertit en una peça**.

Dins dels models grans

Capes convolucionals → **MLP** → Dígit

Les CNN decideixen al final amb capes com aquestes, i cada bloc d'un **transformer** porta un MLP petit.

Softmax, loss, backpropagation i mini-batches hi funcionen igual.

Sobre embeddings

Model ja entrenat → 1.024 números → **MLP**

Un altre model ja ha «mirat» el text o la foto. Un MLP petit només ha de decidir.

Exemple: classificar 2.000 correus etiquetats, en minuts i sense GPU.

En producció

	MLP	FOREST
Encert	97,65%	97,2%
Mida	**1,2 MB**	**126 MB**
Predir 10.000	0,012 s	0,042 s
`partial_fit`	sí	no

A canvi, el random forest s'entrena més de pressa. Temps segons la màquina.

> Com un motor: poca gent en compra un de solt, però gairebé tots els cotxes en porten.

Què faig servir?

SI TENS...	COMENÇA PER...	PER QUÈ
Una taula on cada columna vol dir alguna cosa	Random forest o gradient boosting	Demanen pocs ajustos i s'entrenen de pressa
Imatges	Una CNN (UT6)	Busca el mateix patró a tota la imatge
Text o so	Un model ja entrenat	Començar de zero demana massa dades
Embeddings que ha generat un altre model	Regressió logística o MLP petit	La feina difícil ja està feta
Moltes dades que arriben sense parar, o molt poc espai per al model	**MLP**	`partial_fit` i pocs megabytes

És una guia per començar, no una llei. El mètode no canvia: **provar-ne uns quants i mesurar**.

RESUM

Les set idees que us heu d'endur

1. Una **neurona** fa una suma ponderada, hi suma un biaix i aplica una activació. Separa l'espai amb una recta.
2. El **perceptró** aprèn corregint els pesos quan s'equivoca, però només resol problemes **linealment separables**.
3. Les **capes ocultes** transformen les dades en una representació on el problema sí que és separable.
4. Calen **funcions d'activació** que **doblegin sense salts**, una **loss** i el **gradient descent**.
5. La **backpropagation** reparteix la culpa de l'error cap enrere per saber com moure cada pes.
6. Amb dades reals calen **softmax**, **mini-batches** i **validació**. La capa oculta aprèn característiques, i **l'arquitectura decideix quines** pot aprendre.
7. Avui, un **MLP** és sobretot una **peça**: decideix al final de les CNN, viu dins dels transformers i treballa sobre embeddings.

PER AL DEBAT DE CLASSE

Cinc preguntes sense una única resposta

1. Si una regressió logística és una neurona, per què diem que les xarxes són una **família diferent**?
2. El 1958 es va prometre massa i va venir un hivern. Hi veieu **paral·lelismes amb avui**?
3. Una xarxa encerta el 98% però no s'explica; un decision tree encerta el 94% i es llegeix. Quin triaríeu per **concedir un crèdit**? I per **trobar pingüins en fotos aèries**?
4. La backpropagation és de 1986 i les xarxes no dominen fins al 2012. **Què faltava?**
5. A MNIST, un random forest gairebé iguala l'MLP. Per què, doncs, les xarxes **han guanyat en imatges**?

Conceptes que surten de passada

Aprenentatge incremental `partial_fit`

Continuar entrenant un model amb dades noves sense començar de zero. Un MLP ho permet. Un random forest s'ha de tornar a entrenar sencer, i per això cal conservar totes les dades.

Diap. 40 · NB 1.2 §4.1 i §11.3

Aturada anticipada *early stopping*

Reservar una part de l'entrenament com a validació i aturar quan l'encert de validació deixa de millorar. Evita entrenar epochs de més i caure en l'overfitting. A scikit-learn, `early_stopping=True`.

NB 1.1 ex. 6 · NB 1.2 ex. 5

Augment de dades *data augmentation*

Crear exemples d'entrenament nous modificant els que ja tens: desplaçar, girar o retallar una imatge. La xarxa aprèn que aquests canvis no canvien la classe.

NB 1.2 ex. 7 · UT6

Cerca en graella i validació creuada `GridSearchCV`

La cerca en graella entrena totes les combinacions d'hiperparàmetres d'una llista. Cada una s'avalua amb validació creuada: s'entrena k vegades, deixant cada vegada un tros diferent per avaluar, i se'n fa la mitjana.

NB 1.1 ex. 7 · 5134 UT7

Dades estructurades i no estructurades

Estructurades: una taula on cada columna vol dir alguna cosa (la llargada de l'aleta). No estructurades: imatges, so o text, on una columna sola, un píxel, no diu res. És on destaquen les xarxes.

Diap. 4 · NB 1.1 §12

Decision tree i random forest

Un decision tree és una cadena de preguntes de sí o no sobre les columnes. Un random forest n'entrena molts amb sorteigs diferents de les dades i els fa votar. Amb dades tabulars sovint és el primer que es prova.

Diap. 39 i 41 · NB 1.2 §8 · 5134 UT5

Embedding

Una llista de números, per exemple 1.024, que resumeix un text o una imatge. La genera un model ja entrenat, i amb ella n'hi ha prou amb un model petit i poques dades.

Diap. 40 i 41 · NB 1.2 §11.2

Encert *accuracy*

La proporció de prediccions correctes. És el que retorna `score` d'un classificador. Enganya quan una classe és molt més freqüent que les altres.

Tota la unitat · NB 1.2 §2 · 5134 UT4

Error quadràtic mitjà *MSE*

La mitjana dels errors elevats al quadrat. És la loss habitual en regressió i castiga molt més un error gran que molts de petits. La seva arrel quadrada és el RMSE.

NB 1.1 §7 i §11.1 · 5134 UT2

On surt: diapositiva (Diap. 14), notebook i secció (NB 1.1 §3.2) o exercici (ex. 6). **5134 UT7**: on el treballareu a fons al mòdul germà.

GLOSSARI · 2 DE 4

Conceptes que surten de passada

Escalat `StandardScaler`

Restar la mitjana i dividir per la desviació típica de cada columna, perquè totes tinguin escales semblants. Sense escalar, la columna amb els números més grans domina l'entrenament.

Diap. 14 · NB 1.1 §3.2 · 5134 UT3

GPU

El processador de la targeta gràfica. Fa milers de multiplicacions alhora i entrena xarxes moltes vegades més de pressa que la CPU. A Colab és opcional.

Diap. 4 · NB 1.1 §11 · UT6

Gradient boosting

Algorisme clàssic que encadena decision trees petits, cadascun corregint els errors dels anteriors. Amb dades tabulars sovint iguala o supera les xarxes.

Diap. 4 · NB 1.1 §12 · 5134 UT5

Llavor `random_state`

El número que fixa els sorteigs: pesos inicials, partició, ordre de les mostres. Amb la mateixa llavor surt sempre el mateix resultat; canviant-la, es veu quant depèn de l'atzar.

Diap. 30 i 31 · NB 1.1 §8.2 i §9

Matriu de confusió

Taula de la classe real (files) contra la classe predita (columnes). La diagonal són els encerts, i la resta diu quines classes es confonen entre si.

NB 1.2 §7.1 · 5134 UT4

Mínim local

Una vall que no és la més fonda. El gradient descent s'hi pot quedar encallat, i per això dues llavors diferents poden donar xarxes diferents.

Diap. 30 · NB 1.1 §9

Model de referència `DummyClassifier`

Un model que no aprèn res: prediu sempre la classe més freqüent. Sense aquest punt de comparació, un percentatge d'encert no vol dir res.

Diap. 29 i 39 · NB 1.1 §11 · 5134 UT1

Momentum

Una millora del gradient descent: cada passa recorda la direcció de les anteriors, com una bola que agafa embranzida costa avall. Amb `momentum=0` es desactiva.

NB 1.1 §10.2 · NB 1.2 §4.1

Optimitzador `solver`

El mètode que mou els pesos. `sgd` és el gradient descent de la unitat; `adam`, el de per defecte, ajusta la passa de cada pes; `lbfgs` va bé amb conjunts petits.

Diap. 31 · NB 1.1 §10 i §11

On surt: diapositiva (Diap. 14), notebook i secció (NB 1.1 §3.2) o exercici (ex. 6). **5134 UT7**: on el treballareu a fons al mòdul germà.

Conceptes que surten de passada

Overfitting

El model memoritza les mostres d'entrenament, soroll inclòs, en lloc d'aprendre'n la forma general. L'encert d'entrenament puja i el de validació o test no l'acompanya.

Diap. 30 i 36 · NB 1.1 §10.1 · 5134 UT2

Partició entrenament / test `train_test_split`

Separar una part de les dades que el model no veu en entrenar, per mesurar com funciona amb dades noves. `test_size=0.2` en reserva el 20%.

NB 1.1 §3.2 i §10 · 5134 UT1

Partició estratificada `stratify=y`

Una partició que manté la proporció de cada classe a entrenament i a test. Amb els pingüins (45% Gentoo), el test té sempre 25 Gentoo de 55; a l'atzar, en pot tenir entre 14 i 36.

NB 1.1 §3.2 · 5134 UT7

Pipeline `make_pipeline`

Diversos passos encadenats que es fan servir com un sol model. L'escalador aprèn la mitjana i la desviació només amb l'entrenament i s'aplica sol a les dades noves.

Diap. 14 · NB 1.1 §3.2 · 5134 UT9

Precisió i recall

Precisió: de les vegades que el model diu «sí», quantes té raó. Recall: dels «sí» reals, quants en troba. Diuen més que l'encert quan una classe és rara.

NB 1.2 §2 · 5134 UT4

R2 *coeficient de determinació*

Mètrica de regressió: 1 vol dir perfecte, i 0, tan bo com predir sempre la mitjana. És el que retorna `score` d'un regressor.

NB 1.1 §11.1 · 5134 UT2

Regressió logística `LogisticRegression`

Algorisme clàssic que, malgrat el nom, classifica. Calcula una puntuació per classe directament a partir de les columnes, sense capa oculta, i per això només sap fer fronteres rectes.

Diap. 23, 29 i 37 · NB 1.1 §11 · NB 1.2 §6.2 · 5134 UT4

Regularització `alpha` `C`

Castigar els pesos grans perquè el model faci servir patrons suaus i no memoritzi. A `MLPClassifier`, com més gran és **alpha**, més càstig; a `LogisticRegression`, com més petit és **C**.

NB 1.2 §6.2 · NB 1.1 ex. 7 · 5134 UT4

Saturació

Una neurona amb sigmoide o tanh i una z molt gran o molt petita queda a la part plana de la corba: el pendent és gairebé zero i gairebé no aprèn.

Diap. 26 · NB 1.1 §6.2

On surt: diapositiva (Diap. 14), notebook i secció (NB 1.1 §3.2) o exercici (ex. 6). **5134 UT7**: on el treballareu a fons al mòdul germà.

Conceptes que surten de passada

Soroll

La part de les dades que és atzar i no segueix cap patró: errors de mesura, punts que cauen dins la classe contrària. Un bon model l'ignora; un que fa overfitting el memoritza.

Diap. 30 · NB 1.1 §10

Validació i test segellat

La validació és una part de l'entrenament reservada per prendre decisions: neurones, epochs, model. El test no es mira fins al final, perquè la nota no surti massa optimista.

Diap. 33 i 36 · NB 1.2 §1.3 i §9 · 5134 UT2 i UT7

Transfer learning

Aprofitar el que un model ha après en una tasca per resoldre'n una altra, en lloc de començar de zero. Per exemple, entrenar un model petit sobre els embeddings que genera un model ja entrenat.

NB 1.2 §11.2 · UT6

Valors que falten `NaN` `dropna`

Cel·les buides d'una taula, que pandas mostra com a `NaN`. La majoria de models no les accepten a `fit`, i `dropna()` elimina les files on en falta alguna.

NB 1.1 §3.2 · 5134 UT2

Transformer

La família de models de llenguatge com ChatGPT. Cada bloc té una part que relaciona les paraules entre si i un MLP petit. S'entrena amb les idees d'aquesta unitat: loss, backpropagation i mini-batches.

Diap. 40 · NB 1.2 §11.1

On surt: diapositiva (Diap. 14), notebook i secció (NB 1.1 §3.2) o exercici (ex. 6). **5134 UT7**: on el treballareu a fons al mòdul germà.